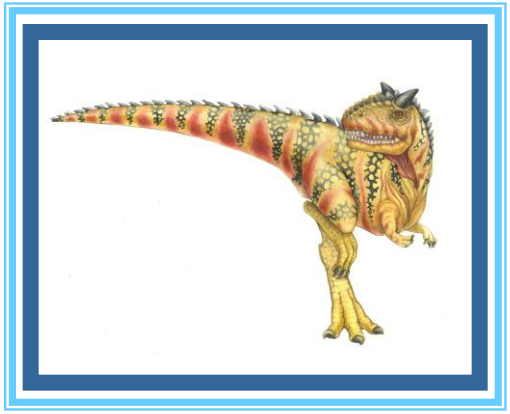
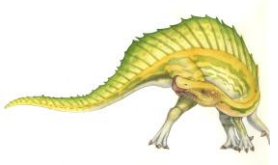
Chapter 14: Protection



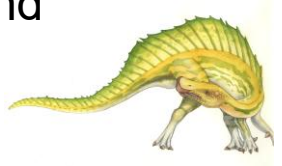
Chapter 14: Protection

- Goals of Protection
- Principles of Protection
- Domain of Protection
- Access Matrix
- Implementation of Access Matrix



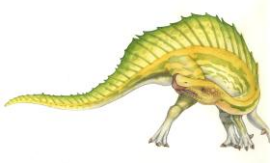
Goals of Protection

- need to ensure that each program component active in a system uses system resources only in ways consistent with stated policies.
- The role of protection in a computer system is to provide a mechanism for the enforcement of the policies governing resource use
- These policies can be established in a variety of ways.
  - Some are fixed in the design of the system,
  - Others are formulated by the management of a system.
  - Still others are defined by the individual users to protect their own files and programs.
- A protection system must have the flexibility to enforce a variety of policies
- In one protection model, computer consists of a collection of objects, hardware or software
- Each object has a unique name and can be accessed through a well-defined set of operations
- Protection problem - ensure that each object is accessed correctly and only by those processes that are allowed to do so



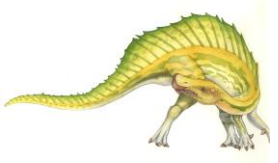
Mechanisms and Policies

- ***mechanisms*** are distinct from ***policies***.
- Mechanisms determine *how* something will be done;
- policies decide *what* will be done. T
- The separation of policy and mechanism is important for flexibility.
- Policies are likely to change from place to place or time to time.
- In the worst case, every change in policy would require a change in the underlying mechanism.
- Using general mechanisms enables us to avoid such a situation.



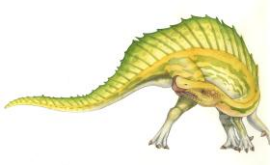
Principles of Protection

- Guiding principle – **principle of least privilege**
  - Programs, users and systems should be given just enough **privileges** to perform their tasks
- An operating system following the **principle of least privilege** implements its features, programs, system calls, and data structures so that failure or compromise of a component does the minimum damage
  - provides system calls and services that allow applications to be written with fine-grained access controls.
  - provides mechanisms to enable privileges when they are needed and to disable them when they are not needed.
  - creation of audit trails for all privileged function access.
  - The audit trail allows the programmer, systems administrator, or law-enforcement officer to trace all protection and security activities on the system



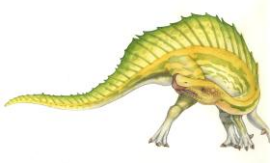
Objects

- A computer system is a collection of processes and objects.
  - **hardware objects** (such as the CPU, memory segments, printers, disks, and tape drives)
  - **software objects** (such as files, programs, and semaphores).
- Each object has a unique name that differentiates it from all other objects in the system, and each can be accessed only through well-defined and meaningful operations.
- Objects are essentially abstract data types
- The operations that are possible may depend on the object.
  - CPU - execute.
  - Memory segments - read and written,
  - whereas a CD-ROM or DVD-ROM- only be read.
  - Tape drives -read,written, and rewound.
  - Data files -created, opened, read, written, closed, and deleted
  - program files - read, written, executed, and deleted



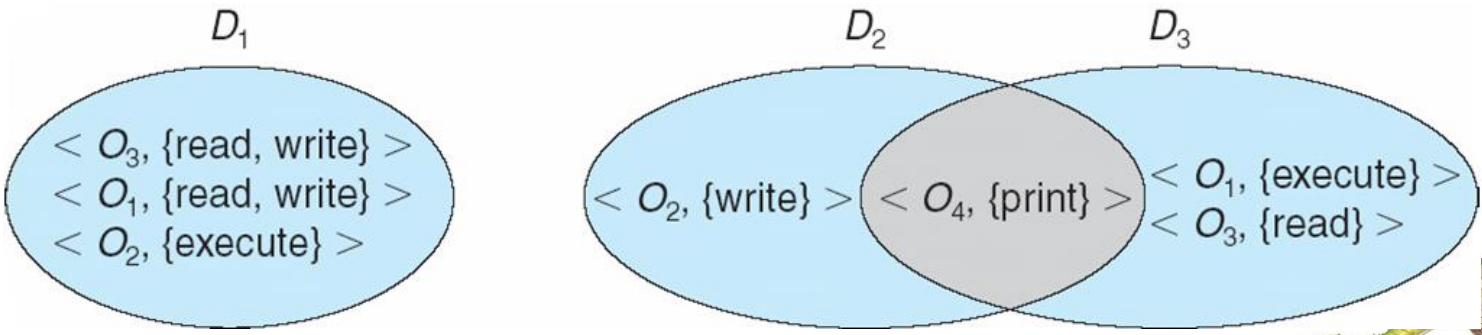
need-to-know principle

- A process should be allowed to access only those resources for which it has authorization.
- At any time, a process should be able to access only those resources that it currently requires to complete its task.
- This second requirement, commonly referred to as the ***need-to-know*** principle, is useful in limiting the amount of damage a faulty process can cause in the system.



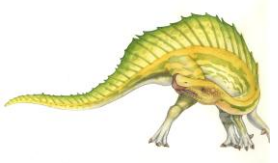
Domain Structure

- process operates within a **protection domain**, which specifies the resources that the process may access.
- Each domain defines a set of objects and the types of operations that may be invoked on each object.
- The ability to execute an operation on an object is an **access right**.
- Access-right = <*object-name*, *rights-set*>
  where *rights-set* - subset of all valid operations that can be performed on the object
- Domain = collection of access-rights
  example, if domain *D* has the access right <*file F, {*read,write*}*>, then a process executing in domain *D* can both read and write file *F*; it cannot, however, perform any other operation on that object.

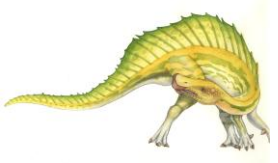


- The association between a process and a domain may be either
  - **Static -**If the set of resources available to the process is fixed throughout the process's lifetime
  - **Dynamic**. - establishing dynamic protection domains is more complicated than establishing static protection domains
- If the association between processes and domains is fixed, and want to adhere to the need-to-know principle, then a mechanism must be available to change the content of a domain.
- a process may execute in two different phases and may, for example, need read access in one phase and write access in another.
- If a domain is static, we must define the domain to include both read and write access.
- this arrangement provides more rights than are needed in each of the two phases, since we have read access in the phase where we need only write access, and vice versa.
- the need-to-know principle is violated.
- Must allow the contents of a domain to be modified so that the domain always reflects the minimum necessary access rights.

- If the association is dynamic, a mechanism is available to allow **domain switching**, enabling the process to switch from one domain to another.
- May also want to allow the content of a domain to be changed.
- If we cannot change the content of a domain, can provide the same effect by creating a new domain with the changed content and switching to that new domain when we want to change the domain content



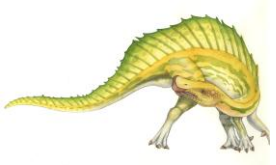
- A domain can be realized in a variety of ways:
- Each *user* may be a domain.
  - the set of objects that can be accessed depends on the identity of the user.
  - Domain switching occurs when the user is changed—generally when one user logs out and another user logs in.
- Each *process* may be a domain.
  - the set of objects that can be accessed depends on the identity of the process.
  - Domain switching occurs when one process sends a message to another process and then waits fora response.
- •Each *procedure* may be a domain.
  - the set of objects that can be accessed corresponds to the local variables defined within the procedure.
  - Domain switching occurs when a procedure call is made.



Access Matrix

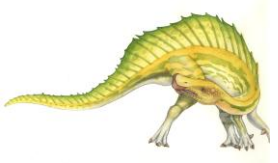
- View protection as a matrix (**access matrix**)
- Rows represent domains
- Columns represent objects
- **`Access(i, j)`** is the set of operations that a process executing in Domain_i can invoke on Object_j

domain \ object	F_1	F_2	F_3	printer
D_1	read		read	
D_2				print
D_3		read	execute	
D_4	read write		read write	



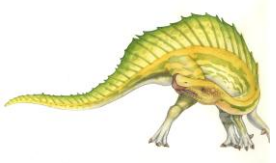
Use of Access Matrix

- If a process in Domain D_i tries to do "op" on object O_j, then "op" must be in the access matrix
- User who creates object can define access column for that object
- When we switch a process fromone domain to another, we are executing an operation (switch) on an object (the domain).
- Can control domain switching by including domains among the objects of the access matrix.
- Similarly, when we change the content of the access matrix, we are performing an operation on an object: the access matrix
- We can control these changes by including the access matrix itself as an object.
- since each entry in the access matrix may be modified individually, we must consider each entry in the access matrix as an object to be protected



Dynamic protection

- Can be expanded to dynamic protection
- Processes should be able to switch from one domain to another.
- Switching from domain *Di* to domain *Dj* is allowed if and only if the access right switch $\in$ access(*i*, *j*).
- a process executing in domain *D*2 can switch to domain *D*3 or to domain *D*4. A process in domain *D*4 can switch to *D*1, and one in domain *D*1 can switch to *D*2.



Access Matrix of Figure A with Domains as Objects

domain \ object	F_1	F_2	F_3	laser printer	D_1	D_2	D_3	D_4
D_1	read		read			switch		
D_2				print			switch	switch
D_3		read	execute					
D_4	read write		read write		switch			

Access Matrix with *Copy* Rights

- Allowing controlled change in the contents of the access-matrix entries requires three additional operations: copy, owner, and control
- The ability to copy an access right from one domain (or row) of the access matrix to another is denoted by an asterisk (*) appended to the access right.
- The *copy* right allows the access right to be copied only within the column (that is, for the object) for which the right is defined

object / domain	F_1	F_2	F_3
D_1	execute		write*
D_2	execute	read*	execute
D_3	execute		

(a)

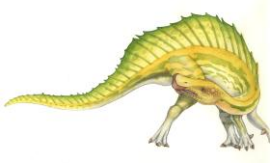
object / domain	F_1	F_2	F_3
D_1	execute		write*
D_2	execute	read*	execute
D_3	execute	read	

- This scheme has two variants:

  1. A right is copied from access(*i*, *j*) to access(*k*, *j*); it is then removed from access(*i*, *j*). This action is a *transfer* of a right, rather than a copy.

  2. Propagation of the *copy* right may be limited. That is, when the right $R*$ is copied from access(*i*, *j*) to access(*k*, *j*), only the right *R* (not $R*$) is created. A process executing in domain *Dk* cannot further copy the right *R*

- A system may select only one of these three *copy* rights, or it may provide all three by identifying them as separate rights: *copy*, *transfer*, and *limited copy*.



Access Matrix With *Owner* Rights

- mechanism to allow addition of new rights and removal of some rights.
- The *owner* right controls these operations.
- If access(i, j) includes the *owner* right, then a process executing in domain *Di* can add and remove any right in any entry in column j. For

domain \ object	F_1	F_2	F_3
D_1	owner execute		write
D_2		read* owner	read* owner write
D_3	execute		

(a)

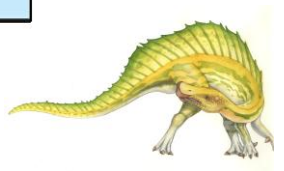
domain \ object	F_1	F_2	F_3
D_1	owner execute		write
D_2		owner read* write*	read* owner write
D_3		write	write

(b)

Modified Access Matrix of Figure B

- The *copy* and *owner* rights allow a process to change the entries in a column.
- A mechanism is also needed to change the entries in a row.
- The *control* right is applicable only to domain objects.
- If access(*i*, *j*) includes the *control* right, then a process executing in domain *Di* can remove any access right from row *j*.

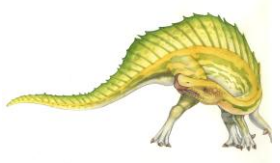
object / domain	F_1	F_2	F_3	laser printer	D_1	D_2	D_3	D_4
D_1	read		read			switch		
D_2				print			switch	switch control
D_3		read	execute					
D_4	write		write		switch			



Implementation of Access Matrix

- Generally, a sparse matrix
- Option 1 – Global table
  - Store ordered triples **<domain, object,rights-set>** in table
  - A requested operation M on object O_j within domain D_i -> search table for < D_i, O_j, R_k >
    - with M ∈ R_k
  - If entry found operation is allowed to continue, else an exception (or error) condition is raised.

  Drawbacks:
  - table could be large -> won't fit in main memory- virtual memory technique needed
  - difficult to take advantage of special groupings of objects or domains.
  - For example, if everyone can read a particular object, this object must have a separate entry in every domain



Implementation of Access Matrix (Cont.)

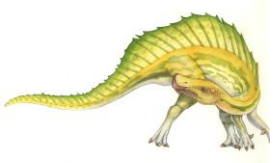
- Option 2 – Access lists for objects
  - Each column implemented as an access list for one object
  - Resulting per-object list consists of ordered pairs **<domain, rights-set>** defining all domains with non-empty set of access rights for the object
  - can be extended easily to define a list plus a *default* set of access rights.
  - When an operation M on an object O_j is attempted in domain D_i, search the access list for object O_j, looking for an entry $<D_i, R_k>$ with $M \in R_k$.
  - If the entry is found, allow the operation; if not, check the default set. If M is in the default set, we allow the access. Otherwise, access is denied, and an exception condition occurs.
  - For efficiency, we may check the default set first and then search the access list.

Implementation of Access Matrix (Cont.)

- Option 3 – Capability list for domains
  - Instead of object-based, list is domain based
  - **Capability list** for domain is list of objects together with operations allows on them
  - Object represented by its name or address, called a **capability**
  - Execute operation M on object O_j, process requests operation and specifies capability as parameter
    - Possession of capability means access is allowed
  - Capability list associated with domain but never directly accessible by domain
    - Rather, protected object, maintained by OS and accessed indirectly
    - Like a "secure pointer"
    - Idea can be extended up to applications

Implementation of Access Matrix (Cont.)

- Option 4 – Lock-key
  - Compromise between access lists and capability lists
  - Each object has list of unique bit patterns, called **locks**
  - Each domain as list of unique bit patterns called **keys**
  - Process in a domain can only access object if domain has key that matches one of the locks
- the list of keys for a domain must be managed by the operating system on behalf of the domain. Users are not allowed to
- examine or modify the list of keys (or locks) directly.

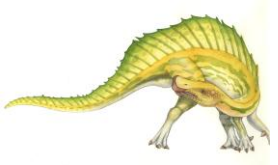


Comparison of Implementations

- Many trade-offs to consider
  - Global table is simple, but can be large
  - Access lists correspond to needs of users
    - When a user creates an object, he can specify which domains can access the object, as well as what operations are allowed.
    - because access-rights information for a particular domain is not localized, determining the set of access rights for each domain is difficult.
    - every access to the object must be checked, requiring a search of the access list.
    - In a large system with long access lists, this search can be time consuming.
  - Capability lists useful for localizing information for a given process
    - do not correspond directly to the needs of users
    - revocation capabilities can be inefficient
    - protection system needs only to verify that the capability is valid
  - Lock-key effective and flexible, keys can be passed freely from domain to domain, easy revocation

Comparison of Implementations (Cont.)

- Most systems use combination of access lists and capabilities
  - First access to an object -> access list searched
    - If allowed, capability created and attached to process
      - Additional accesses need not be checked
    - After last access, capability destroyed



Reference

- A. Silberschatz, P. B. Galvin and G. Gagne, *Operating System Concepts*, (9e), Wiley and Sons (Asia) Pvt. Ltd, 2016.